Prompt

generate C++ code using objects, not functions, to implement
the following requirements

• Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.

• A typical puzzle is shown here (image credit: Wikimedia Commons):

o

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:

o Each digit from 1 to 9 must appear exactly once in each row.

o Each digit from 1 to 9 must appear exactly once in each column.

o Each digit from 1 to 9 must appear exactly once in each block.

• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.

• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.

• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it’s possible to gradually fill in the entire grid.

• That is not the approach we’re going to use. Instead, we’re going to

implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by “slower to run,” we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
• Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five puzzle files.
• The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
• To help you debug your program, solution1.txt is a solution to puzzle1.txt.
(all puzzle.txt's attached to both ais)

Gemini

Code

```
/*
* Program Name: EECS 348 Assignment 4
```

```
*
* Description: A program that solves sudoku puzzles using a recursive object
oriented searching method. It finds every solution showing puzzles that have
multiple solutions and showing puzzles with no solution
*
* Inputs: Text files that contain 9x9 sudoku puzzles, a row per line that
contain digits from 1-9 for filled pieces and _ for empty pieces. Uses the
files puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
*
* Outputs: The file name, the puzzle read from the file, and then the
solutions all written to the output
*
* Collaborators: Connor Wiley
*
* Other sources: Code generated via Google Gemini 3
*
* Author: Connor Wiley, Gemini
*
* Creation Date: 10/15/2026
*/



#include <iostream>
#include <fstream>
#include <vector>
#include <string>


class SudokuPuzzle {

private:
    std::vector<std::vector<int>> grid;
    std::string filename;


    // Checks if placing a number is valid according to Sudoku constraints

    bool isValid(int row, int col, int num) const {

        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == num) return false;
            if (grid[i][col] == num) return false;
        }

        int startRow = row - row % 3;
```

```
    int startCol = col - col % 3;

    for (int r = 0; r < 3; ++r) {
        for (int c = 0; c < 3; ++c) {
            if (grid[startRow + r][startCol + c] == num) return false;
        }
    }
    return true;
}



// Finds the next empty cell (represented by 0)

bool findEmptyCell(int& row, int& col) const {
    for (row = 0; row < 9; ++row) {
        for (col = 0; col < 9; ++col) {
            if (grid[row][col] == 0) {
                return true;
            }
        }
    }
    return false;
}



// Recursive backtracking method to find and print all solutions

void solveRecursive(int& solutionCount) {
    int row, col;
    if (!findEmptyCell(row, col)) {
        // Puzzle successfully solved
        solutionCount++;
        std::cout << "Solution " << solutionCount << ":\n";
        printGrid();
        std::cout << "\n";
        return;
    }


    for (int num = 1; num <= 9; ++num) {
        if (isValid(row, col, num)) {
            grid[row][col] = num;
            solveRecursive(solutionCount);
            grid[row][col] = 0; // Backtrack
```

```
            }
        }
    }



public:
    // Constructor initializes the grid and stores the filename
    SudokuPuzzle(const std::string& fname) : filename(fname), grid(9,
std::vector<int>(9, 0)) {}

    // Loads the puzzle grid from the file
    bool load() {
        std::ifstream file(filename);
        if (!file.is_open()) return false;


        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                char ch;
                if (!(file >> ch)) return false;
                if (ch == '_') {
                    grid[r][c] = 0;
                } else if (ch >= '1' && ch <= '9') {
                    grid[r][c] = ch - '0';
                }
            }
        }
        return true;
    }



    // Prints the Sudoku grid with underscores for blanks

    void printGrid() const {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                if (grid[r][c] == 0) {
                    std::cout << "_ ";
                } else {
                    std::cout << grid[r][c] << " ";
                }
            }
            std::cout << "\n";
        }
```

```
    }



    // Solves the puzzle and prints outputs matching assignment specifications

    void solve() {
        std::cout << "Puzzle File: " << filename <<[cite: 2] << "\n";
        std::cout << "Initial Puzzle:\n";
        printGrid();
        std::cout << "\n";


        int solutionCount = 0;
        solveRecursive(solutionCount);


        if (solutionCount == 0) {
            std::cout << "No solution found\n"[cite: 2];
        } else {
            std::cout << "Total solutions found: " << solutionCount << "\n";
        }
        std::cout << "========================================\n\n";
    }
};



int main() {
    // Array containing the five puzzle files provided
    std::string puzzleFiles[] = {
        "puzzle1.txt",[cite: 7]
        "puzzle2.txt",[cite: 6]
        "puzzle3.txt",[cite: 5]
        "puzzle4.txt",[cite: 4]
        "puzzle5.txt" [cite: 3]
    };



    for (const std::string& fname : puzzleFiles) {
        SudokuPuzzle puzzle(fname);
        if (puzzle.load()) {
            puzzle.solve();
        } else {
            std::cerr << "Error: Could not open file " << fname << "\n";
```

```
        }
    }


    return 0;
}
```

Analysis

```
/*
* Program Name: EECS 348 Assignment 4
*
* Description: A program that solves sudoku puzzles using a recursive object
oriented searching method. It finds every solution showing puzzles that have
multiple solutions and showing puzzles with no solution
*
* Inputs: Text files that contain 9x9 sudoku puzzles, a row per line that
contain digits from 1-9 for filled pieces and _ for empty pieces. Uses the
files puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
*
* Outputs: The file name, the puzzle read from the file, and then the
solutions all written to the output
*
* Collaborators: Connor Wiley
*
* Other sources: Code generated via Google Gemini 3
*
* Author: Connor Wiley, Gemini
*
* Creation Date: 10/15/2026
*/

#include <iostream> // allows usage of cout and cerr, used for printing the
puzzles, solutions, and error messages
#include <fstream> // allows usage of ifstream, used to open the puzzle files
#include <vector> // allows usage of vector type, used to hold the 9x9 grid
#include <string> // allows usage of the string type, used for filenames

class SudokuPuzzle { // creates a class SudokuPuzzle that holds a puzzle and
solves it
private: // makes it so everything below can only be used inside the class
    std::vector<std::vector<int>> grid; // instantiates a 2D vector of ints
named grid that holds the 9x9 board
    std::string filename; // instantiates a string named filename that holds
the name of the puzzle file
```

```
    // Checks if placing a number is valid according to Sudoku constraints
    bool isValid(int row, int col, int num) const { // creates the isValid
function that checks if a number is legal to place at a row and column
        for (int i = 0; i < 9; ++i) { // loops through all 9 positions to
check the row and column at the same time
            if (grid[row][i] == num) return false; // checks the row for the
same number, if theres a match its invalid
            if (grid[i][col] == num) return false; // checks the column for
the same number, if theres a match its invalid
        } // ends for loop
        int startRow = row - row % 3; // instantiates an int named startRow
for the top row of the 3x3 block this cell is in
        int startCol = col - col % 3; // instantiates an int named startCol
for the left column of the 3x3 block this cell is in
        for (int r = 0; r < 3; ++r) { // loops through the 3 rows of the block
            for (int c = 0; c < 3; ++c) { // loops through the 3 columns of
the block
                if (grid[startRow + r][startCol + c] == num) return false; //
checks the block for the same number, if theres a match its invalid
            } // ends the for loop for columns
        } // ends the for loop for rows
        return true; // returns true since nothing clashed so the number is
allowed
    } // ends isValid function

    // Finds the next empty cell (represented by 0)
    bool findEmptyCell(int& row, int& col) const { // creates the
findEmptyCell function that finds the next empty cell, row and col are
references so the answer gets sent back through them
        for (row = 0; row < 9; ++row) { // loops through each row, using the
reference itself as the counter
            for (col = 0; col < 9; ++col) { // loops through each column,
using the reference itself as the counter
                if (grid[row][col] == 0) { // checks if the cell is empty
                    return true; // returns true since an empty cell was
found, row and col are left pointing at it
                } // ends if statement
            } // ends the for loop for columns
        } // ends the for loop for rows
        return false; // returns false since no empty cells are left, meaning
the board is full
    } // ends findEmptyCell function

    // Recursive backtracking method to find and print all solutions
    void solveRecursive(int& solutionCount) { // creates the solveRecursive
```

```
function that fills the next empty cell, calls itself again, and prints every
solution it finds, solutionCount is a reference so the count carries through
every call
        int row, col; // instantiates ints named row and col that hold the
location of the next empty cell
        if (!findEmptyCell(row, col)) { // checks if there are no empty cells
left
            // Puzzle successfully solved
            solutionCount++; // adds 1 to the solution count
            std::cout << "Solution " << solutionCount << ":\n"; // prints the
numbered label for this solution
            printGrid(); // prints the solved board
            std::cout << "\n"; // prints a blank line after the board
            return; // goes back to the previous call to try other options
        } // ends if statement

        for (int num = 1; num <= 9; ++num) { // loops through the digits 1
through 9
            if (isValid(row, col, num)) { // checks if the digit is legal in
that spot
                grid[row][col] = num; // puts the digit in the cell
                solveRecursive(solutionCount); // calls solveRecursive again
now that the cell is filled
                grid[row][col] = 0; // Backtrack // sets the cell back to
empty so the next digit can be tried
            } // ends if statement
        } // ends for loop
    } // ends solveRecursive function

public: // makes it so everything below can be called by outside functions
    // Constructor initializes the grid and stores the filename
    SudokuPuzzle(const std::string& fname) : filename(fname), grid(9,
std::vector<int>(9, 0)) {} // creates a constructor that stores the filename
and makes the grid 9 rows of 9 zeros so we have an empty board

    // Loads the puzzle grid from the file
    bool load() { // creates the load function that fills the grid from the
file and returns whether it worked
        std::ifstream file(filename); // instantiates an ifstream named file
that opens the puzzle file
        if (!file.is_open()) return false; // checks if the file failed to
open, if so returns false

        for (int r = 0; r < 9; ++r) { // loops through each row
            for (int c = 0; c < 9; ++c) { // loops through each column
                char ch; // instantiates a char named ch that holds each
```

```
character as it is read
                if (!(file >> ch)) return false; // reads the next non
whitespace character, if the file ran out too early returns false
                if (ch == '_') { // checks if the character represents a blank
cell
                    grid[r][c] = 0; // sets that cell to 0
                } else if (ch >= '1' && ch <= '9') { // checks if the
character is a digit from 1 to 9
                    grid[r][c] = ch - '0'; // sets that cell to the digit,
subtracting '0' converts the character into an int
                } // ends if statement
            } // ends the for loop for columns
        } // ends the for loop for rows
        return true; // returns true since all 81 cells were read
    } // ends load function

    // Prints the Sudoku grid with underscores for blanks
    void printGrid() const { // creates the printGrid function that prints the
board with underscores for the blanks
        for (int r = 0; r < 9; ++r) { // loops through each row
            for (int c = 0; c < 9; ++c) { // loops through each column
                if (grid[r][c] == 0) { // checks if the cell is empty
                    std::cout << "_ "; // prints an underscore and a space
                } else { // ends the if statement but runs if the cell has a
digit
                    std::cout << grid[r][c] << " "; // prints the digit and a
space
                } // ends else statement
            } // ends the for loop for columns
            std::cout << "\n"; // moves to the next line after each row
        } // ends the for loop for rows
    } // ends printGrid function

    // Solves the puzzle and prints outputs matching assignment specifications
    void solve() { // creates the solve function that prints the puzzle,
solves it, and prints the results
        std::cout << "Puzzle File: " << filename << "\n"; // prints which file
is being used
        std::cout << "Initial Puzzle:\n"; // prints the label for the starting
board
        printGrid(); // prints the starting board
        std::cout << "\n"; // prints a blank line after the board

        int solutionCount = 0; // instantiates an int named solutionCount set
to 0 that tracks how many solutions were found
        solveRecursive(solutionCount); // starts the recursive search, which
```

```
prints each solution as it finds it

        if (solutionCount == 0) { // checks if no solutions were found
            std::cout << "No solution found\n"; // prints that there is no
solution
        } else { // ends the if statement but runs if at least one solution
was found
            std::cout << "Total solutions found: " << solutionCount << "\n";
// prints how many solutions were found
        } // ends else statement
        std::cout << "========================================\n\n"; // prints
a line to separate the puzzles
    } // ends solve function
}; // ends SudokuPuzzle class

int main() { // instantiates the main function for running the program
    // Array containing the five puzzle files provided
    std::string puzzleFiles[] = { // instantiates an array of strings named
puzzleFiles that holds the filenames
        "puzzle1.txt", // the first puzzle file
        "puzzle2.txt", // the second puzzle file
        "puzzle3.txt", // the third puzzle file
        "puzzle4.txt", // the fourth puzzle file
        "puzzle5.txt" // the fifth puzzle file
    }; // ends the array

    for (const std::string& fname : puzzleFiles) { // loops through each
filename
        SudokuPuzzle puzzle(fname); // creates a SudokuPuzzle for that file
        if (puzzle.load()) { // checks if the puzzle loaded from the file
correctly
            puzzle.solve(); // solves the puzzle and prints the results
        } else { // ends the if statement but runs if the file couldnt be
loaded
            std::cerr << "Error: Could not open file " << fname << "\n"; //
prints an error message with the filename
        } // ends else statement
    } // ends for loop

    return 0; // returns 0 for the main
} // ends main function
```

Claude

Code

```
/*
* Program Name: EECS 348 Assignment 4
*
* Description: A program that solves sudoku puzzles using a recursive object
oriented searching method. It finds every solution showing puzzles that have
multiple solutions and showing puzzles with no solution
*
* Inputs: Text files that contain 9x9 sudoku puzzles, a row per line that
contain digits from 1-9 for filled pieces and _ for empty pieces. Uses the
files puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
*
* Outputs: The file name, the puzzle read from the file, and then the
solutions all written to the output
*
* Collaborators: Connor Wiley
*
* Other sources: Code generated via Claude Anthropic Sonnet 5.5
*
* Author: Connor Wiley, Claude Anthropic
*
* Creation Date: 10/15/2026
*/



// Sudoku brute-force solver (depth-first search with backtracking).

// Object-oriented design: all logic lives in classes; main() only creates

// an application object and runs it.

//

// Build:  g++ -std=c++17 -O2 -o sudoku sudoku.cpp

// Run:    ./sudoku puzzle1.txt puzzle2.txt puzzle3.txt puzzle4.txt
puzzle5.txt

//         (with no arguments it looks for puzzle1.txt ... puzzle5.txt in the

//          current directory)



#include <array>
#include <cctype>
```

```
#include <fstream>
#include <iostream>
#include <string>
#include <vector>



// ----------------------------------------------------------------------------

// Board: a 9x9 grid of digits (0 = empty). Knows the Sudoku rules.

// ----------------------------------------------------------------------------

class Board {
public:
    static const int SIZE = 9;
    static const int BLOCK = 3;


    Board() { cells_.fill(0); }


    int get(int row, int col) const { return cells_[row * SIZE + col]; }
    void set(int row, int col, int value) { cells_[row * SIZE + col] = value;
}



    // True if `value` can legally be placed at (row, col) given the other
cells.

    bool isCandidate(int row, int col, int value) const {
        for (int i = 0; i < SIZE; ++i) {
            if (i != col && get(row, i) == value) return false;  // row
            if (i != row && get(i, col) == value) return false;  // column
        }

        int br = (row / BLOCK) * BLOCK, bc = (col / BLOCK) * BLOCK;
        for (int r = br; r < br + BLOCK; ++r)
            for (int c = bc; c < bc + BLOCK; ++c)
                if ((r != row || c != col) && get(r, c) == value) return
false;  // block
        return true;
    }
```

```
    // List of digits that may legally go into (row, col).

    std::vector<int> candidates(int row, int col) const {
        std::vector<int> result;
        for (int v = 1; v <= SIZE; ++v)
            if (isCandidate(row, col, v)) result.push_back(v);
        return result;
    }



    // Finds the next empty cell in row-major order. Returns false if the
board is full.

    bool findEmpty(int& row, int& col) const {
        for (int i = 0; i < SIZE * SIZE; ++i) {
            if (cells_[i] == 0) {
                row = i / SIZE;
                col = i % SIZE;
                return true;
            }
        }
        return false;
    }



    // True if no filled cell conflicts with another (used to vet the given
puzzle).

    bool isConsistent() const {
        for (int r = 0; r < SIZE; ++r)
            for (int c = 0; c < SIZE; ++c) {
                int v = get(r, c);
                if (v != 0 && !isCandidate(r, c, v)) return false;
            }
        return true;
    }



    void print(std::ostream& out) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                int v = get(r, c);
```

```
                out << (v == 0 ? '_' : static_cast<char>('0' + v)) << ' ';
            }
            out << '\n';
        }
    }



private:
    std::array<int, SIZE * SIZE> cells_;
};



// ----------------------------------------------------------------------------

// PuzzleReader: loads a Board from a text file ('_' = blank).

// ----------------------------------------------------------------------------

class PuzzleReader {
public:
    // Returns true on success and fills `board`.
    bool read(const std::string& filename, Board& board) const {
        std::ifstream in(filename);
        if (!in) return false;


        int count = 0;
        char ch;
        while (in.get(ch) && count < Board::SIZE * Board::SIZE) {
            if (ch == '_' || ch == '.' || ch == '0') {
                board.set(count / Board::SIZE, count % Board::SIZE, 0);
                ++count;
            } else if (ch >= '1' && ch <= '9') {
                board.set(count / Board::SIZE, count % Board::SIZE, ch - '0');
                ++count;
            }  // whitespace and anything else is ignored
        }
        return count == Board::SIZE * Board::SIZE;
    }
};



// ----------------------------------------------------------------------------
```

```
// Solver: recursive depth-first search with backtracking. Collects EVERY

// solution (the puzzle may not have a unique one).

// ----------------------------------------------------------------------------

class Solver {
public:
    // Solves `puzzle` and returns all solutions found (empty => no solution).
    std::vector<Board> solveAll(const Board& puzzle) {
        solutions_.clear();
        if (!puzzle.isConsistent()) return solutions_;  // given digits
already clash
        Board work = puzzle;
        search(work);
        return solutions_;
    }


private:
    std::vector<Board> solutions_;


    // Recursive step: fill the next empty cell with each candidate in turn.

    void search(Board& board) {
        int row, col;
        if (!board.findEmpty(row, col)) {  // k == 81: every cell filled
            solutions_.push_back(board);
            return;
        }
        for (int value : board.candidates(row, col)) {  // none => dead end,
backtrack
            board.set(row, col, value);
            search(board);          // recurse with one more cell filled
            board.set(row, col, 0); // undo and try the next candidate
        }
    }
};


// ----------------------------------------------------------------------------

// Application: drives the whole program (reading, solving, reporting).
```

```
// ----------------------------------------------------------------------------

class SudokuApplication {
public:
    int run(const std::vector<std::string>& files) {
        int status = 0;
        for (const std::string& name : files) {
            std::cout << "============================\n";
            std::cout << "Puzzle file: " << name << "\n\n";

            Board puzzle;
            if (!reader_.read(name, puzzle)) {
                std::cout << "Could not read a valid puzzle from this
file.\n\n";
                status = 1;
                continue;
            }

            std::cout << "Puzzle:\n";
            puzzle.print(std::cout);
            std::cout << '\n';

            std::vector<Board> solutions = solver_.solveAll(puzzle);
            if (solutions.empty()) {
                std::cout << "No solution found\n\n";
            } else {
                std::cout << solutions.size()
                          << (solutions.size() == 1 ? " solution found:\n\n" :
" solutions found:\n\n");
                for (std::size_t i = 0; i < solutions.size(); ++i) {
                    if (solutions.size() > 1) std::cout << "Solution " << (i
1) << ":\n";
                    else std::cout << "Solution:\n";
                    solutions[i].print(std::cout);
                    std::cout << '\n';
                }
            }
        }
        return status;
    }

private:

   PuzzleReader reader_;

   Solver solver_;
```

```
};



int main(int argc, char* argv[]) {
    std::vector<std::string> files;
    for (int i = 1; i < argc; ++i) files.push_back(argv[i]);
    if (files.empty())
        for (int i = 1; i <= 5; ++i) files.push_back("puzzle" +
std::to_string(i) + ".txt");

    SudokuApplication app;
    return app.run(files);
}
```

Analysis

```
/*
* Program Name: EECS 348 Assignment 4
*
* Description: A program that solves sudoku puzzles using a recursive object
oriented searching method. It finds every solution showing puzzles that have
multiple solutions and showing puzzles with no solution
*
* Inputs: Text files that contain 9x9 sudoku puzzles, a row per line that
contain digits from 1-9 for filled pieces and _ for empty pieces. Uses the
files puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
*
* Outputs: The file name, the puzzle read from the file, and then the
solutions all written to the output
*
* Collaborators: Connor Wiley
*
* Other sources: Code generated via Claude Anthropic Sonnet 5.5
*
* Author: Connor Wiley, Claude Anthropic
*
* Creation Date: 10/15/2026
*/



// Sudoku brute-force solver (depth-first search with backtracking).

// Object-oriented design: all logic lives in classes; main() only creates
```

```
// an application object and runs it.

//

// Build:  g++ -std=c++17 -O2 -o sudoku sudoku.cpp

// Run:    ./sudoku puzzle1.txt puzzle2.txt puzzle3.txt puzzle4.txt
puzzle5.txt

//         (with no arguments it looks for puzzle1.txt ... puzzle5.txt in the

//          current directory)



#include <array> // allows usage of std::array, used as a fixed size for the
board

#include <cctype> // allows usage of character helper functions

#include <fstream> // allows usage of ifstream, used to open the puzzle files

#include <iostream> // allows usage of cout and ostream, used for printing
outputs like boards and messages

#include <string> // allows usage of the string type, used for filenames

#include <vector> // allows usage of vector type, used to hold solutions,
filenames, and the potential solutions



// ----------------------------------------------------------------------------

// Board: a 9x9 grid of digits (0 = empty). Knows the Sudoku rules.

// ----------------------------------------------------------------------------

class Board { // creates a class Board that holds grids and defines sudoku

public: // makes it so everything below can be called by outside functions

    static const int SIZE = 9; // instantiates an int named size for the width
and height of the board, static so it doesnt change
```

```
    static const int BLOCK = 3; // instantiates an int named block for the
width and height of each small box 3x3 for example



    Board() { cells_.fill(0); } // creates a constructor that fills each cell
with zero so we have an empty board



    int get(int row, int col) const { return cells_[row * SIZE + col]; } //
creates the get function that returns the value at a row and column in the
array

    void set(int row, int col, int value) { cells_[row * SIZE + col] = value;
} // creates the set function that puts a value at the specified row and
column in the array.



    // True if `value` can legally be placed at (row, col) given the other
cells.

    bool isCandidate(int row, int col, int value) const { // creates the
isCandiate function that checks if a spot is legal to enter

        for (int i = 0; i < SIZE; ++i) { // loops through all positions in the
row and columns to check them

            if (i != col && get(row, i) == value) return false;  // row //
checks the rows

            if (i != row && get(i, col) == value) return false;  // column //
checks the columns

        } // ends the for loop for columns and rows

        int br = (row / BLOCK) * BLOCK, bc = (col / BLOCK) * BLOCK; //
instantiates the top left corner of the 3x3 block the cells is in, rounding it
down

        for (int r = br; r < br + BLOCK; ++r) // loops through the 3 rows of
the block

            for (int c = bc; c < bc + BLOCK; ++c) // loops through the 3
columns of the block
```

```
                if ((r != row || c != col) && get(r, c) == value) return
false;  // block // checks the block for exact value, skipping the original
cell, if theres a match its invalid

        return true; // combonation is correct so value is allowed

    } // ends isCanidate function



    // List of digits that may legally go into (row, col).

    std::vector<int> candidates(int row, int col) const { // creates the
canidates function that returns the digits that could go in a location

        std::vector<int> result; // instantiates a vector of ints named result
that holds the digits that are legal

        for (int v = 1; v <= SIZE; ++v) // loops through the digits 1 through
9

            if (isCandidate(row, col, v)) result.push_back(v); // checks if
the digit is legal in that spot, if so adds it to result

        return result; // returns the vector of legal digits, empty if none
work

    } // ends candidates function



    // Finds the next empty cell in row-major order. Returns false if the
board is full.

    bool findEmpty(int& row, int& col) const { // creates the findEmpty
function that finds the next empty cell, row and col are references so the
answer gets sent back through them

        for (int i = 0; i < SIZE * SIZE; ++i) { // loops through all 81 cells
in order

            if (cells_[i] == 0) { // checks if the cell is empty

                row = i / SIZE; // converts the index into a row number
```

```
                col = i % SIZE; // converts the index into a column number

                return true; // returns true since an empty cell was found

            } // ends if statement

        } // ends for loop

        return false; // returns false since no empty cells are left, meaning
the board is full

    } // ends findEmpty function



    // True if no filled cell conflicts with another (used to vet the given
puzzle).

    bool isConsistent() const { // creates the isConsistent function that
checks if the numbers already on the board break any rules

        for (int r = 0; r < SIZE; ++r) // loops through each row

            for (int c = 0; c < SIZE; ++c) { // loops through each column

                int v = get(r, c); // instantiates an int named v that holds
the value at that cell

                if (v != 0 && !isCandidate(r, c, v)) return false; // checks
if the cell is filled and its value conflicts with another cell, if so returns
false

            } // ends the for loop for columns

        return true; // returns true since nothing conflicted

    } // ends isConsistent function



    void print(std::ostream& out) const { // creates the print function that
writes the board to whatever output stream it is given

        for (int r = 0; r < SIZE; ++r) { // loops through each row

            for (int c = 0; c < SIZE; ++c) { // loops through each column
```

```
                int v = get(r, c); // instantiates an int named v that holds
the value at that cell

                out << (v == 0 ? '_' : static_cast<char>('0' + v)) << ' '; //
prints an underscore if the cell is empty, otherwise prints the digit as a
character, then a space

            } // ends the for loop for columns

            out << '\n'; // moves to the next line after each row

        } // ends the for loop for rows

    } // ends print function



private: // makes it so everything below can only be used inside the class

    std::array<int, SIZE * SIZE> cells_; // instantiates an array of 81 ints
named cells_ that stores the whole board in one line

}; // ends Board class



// ----------------------------------------------------------------------------

// PuzzleReader: loads a Board from a text file ('_' = blank).

// ----------------------------------------------------------------------------

class PuzzleReader { // creates a class PuzzleReader that loads a board from a
text file

public: // makes it so everything below can be called by outside functions

    // Returns true on success and fills `board`.

    bool read(const std::string& filename, Board& board) const { // creates
the read function that fills a board from a file and returns whether it worked

        std::ifstream in(filename); // instantiates an ifstream named in that
opens the file
```

```
        if (!in) return false; // checks if the file failed to open, if so
returns false

        // Correctness: extra training content isnt checked and is accepted
with no warning and is able to shift the grid, also doesnt verify that its a
9x9 grid

        int count = 0; // instantiates an int named count that tracks how many
cells have been filled

        char ch; // instantiates a char named ch that holds each character as
it is read

        while (in.get(ch) && count < Board::SIZE * Board::SIZE) { // loops
reading one character at a time until the file ends or all 81 cells are filled

            if (ch == '_' || ch == '.' || ch == '0') { // checks if the
character represents a blank cell

                board.set(count / Board::SIZE, count % Board::SIZE, 0); //
sets that cell to 0, count gets converted into a row and column

                ++count; // adds 1 to count to move to the next cell

            } else if (ch >= '1' && ch <= '9') { // checks if the character is
a digit from 1 to 9

                board.set(count / Board::SIZE, count % Board::SIZE, ch - '0');
// sets that cell to the digit, subtracting '0' converts the character into an
int

                ++count; // adds 1 to count to move to the next cell

            }  // whitespace and anything else is ignored // ends the else if
statement

        } // ends while statement

        return count == Board::SIZE * Board::SIZE; // returns true only if
exactly 81 cells were filled

    } // ends read function

}; // ends PuzzleReader class
```

```
// ---------------------------------------------------------------------------

// Solver: recursive depth-first search with backtracking. Collects EVERY

// solution (the puzzle may not have a unique one).

// ---------------------------------------------------------------------------

class Solver { // creates a class Solver that finds every solution using
backtracking

public: // makes it so everything below can be called by outside functions

    // Solves `puzzle` and returns all solutions found (empty => no solution).

    std::vector<Board> solveAll(const Board& puzzle) { // creates the solveAll
function that returns every solution to a puzzle

        solutions_.clear(); // clears any solutions left over from a previous
puzzle

        if (!puzzle.isConsistent()) return solutions_;  // given digits
already clash // checks if the given numbers already break the rules, if so
returns the empty list since theres no solution

        Board work = puzzle; // instantiates a Board named work that is a copy
of the puzzle so the original doesnt change

        search(work); // calls search on the copy to start finding solutions

        return solutions_; // returns every solution that was found

    } // ends solveAll function




private: // makes it so everything below can only be used inside the class

    // Space complexity: every solution is stored (324 bytes each as ints), so
memory grows as O(S) with the number of solutions S, with no cap. A nearly
empty puzzle has an astronomical number of solutions and would exhaust memory.
```

```
    std::vector<Board> solutions_; // instantiates a vector of Boards named
solutions_ that holds each finished board

    // Execution time: worst case for execution time is O(9^m) for m empty
cells and these nodes are expensive, it also rescans all of the cells on
findEmpty() running, these pair for a long and expensive runtime when the
input doesn't have many cells filled in.

    // Recursive step: fill the next empty cell with each candidate in turn.

    void search(Board& board) { // creates the search function that fills the
next empty cell and then calls itself again

        int row, col; // instantiates ints named row and col that hold the
location of the next empty cell

        if (!board.findEmpty(row, col)) {  // checks if there are no empty
cells left

            solutions_.push_back(board); // adds the finished board to the
list of solutions

            return; // goes back to the previous call to try other options

        } // ends if statement

        for (int value : board.candidates(row, col)) {  // none => dead end,
backtrack // loops through each legal digit for that cell

            board.set(row, col, value); // puts the digit in the cell

            search(board);          // recurse with one more cell filled //
calls search again now that the cell is filled

            board.set(row, col, 0); // undo and try the next candidate // sets
the cell back to empty so the next digit can be tried

        } // ends for loop

    } // ends search function

}; // ends Solver class



// ------------------------------------------------------------------------
```

```
// Application: drives the whole program (reading, solving, reporting).

// ----------------------------------------------------------------------

class SudokuApplication { // creates a class SudokuApplication that runs the whole program

public: // makes it so everything below can be called by outside functions

    int run(const std::vector<std::string>& files) { // creates the run function that goes through each puzzle file and returns 1 if any of them couldnt be read

        int status = 0; // instantiates an int named status set to 0, meaning everything has worked so far

        for (const std::string& name : files) { // loops through each filename

            std::cout << "==============================\n"; // prints a line to separate the puzzles

            std::cout << "Puzzle file: " << name << "\n\n"; // prints which file is being used



            Board puzzle; // instantiates an empty Board named puzzle to read the file into

            if (!reader_.read(name, puzzle)) { // checks if the file couldnt be read into a valid board

                std::cout << "Could not read a valid puzzle from this file.\n\n"; // prints an error message

                status = 1; // sets status to 1 so main knows something failed

                continue; // skips to the next file

            } // ends if statement



            std::cout << "Puzzle:\n"; // prints the label for the starting board
```

```
            puzzle.print(std::cout); // prints the starting board

            std::cout << '\n'; // prints a blank line after the board



            std::vector<Board> solutions = solver_.solveAll(puzzle); //
instantiates a vector of Boards named solutions that holds everything the
solver finds

            if (solutions.empty()) { // checks if no solutions were found

                std::cout << "No solution found\n\n"; // prints that there is
no solution

            } else { // ends the if statement but runs if at least one
solution was found

                std::cout << solutions.size()

                          << (solutions.size() == 1 ? " solution found:\n\n" :
" solutions found:\n\n"); // prints how many solutions were found, using
singular or plural depending on the count

                for (std::size_t i = 0; i < solutions.size(); ++i) { // loops
through each solution

                    if (solutions.size() > 1) std::cout << "Solution " << (i +
1) << ":\n"; // checks if theres more than one solution, if so prints a
numbered label

                    else std::cout << "Solution:\n"; // otherwise prints the
plain label

                    solutions[i].print(std::cout); // prints the solved board

                    std::cout << '\n'; // prints a blank line after the board

                } // ends for loop

            } // ends else statement

        } // ends for loop

        return status; // returns 0 if every file was read fine, otherwise 1
```

```
    } // ends run function



private: // makes it so everything below can only be used inside the class

    PuzzleReader reader_; // instantiates a PuzzleReader named reader_ that
loads the puzzle files

    Solver solver_; // instantiates a Solver named solver_ that solves the
puzzles

}; // ends SudokuApplication class



int main(int argc, char* argv[]) { // instantiates the main function for
running the program

    std::vector<std::string> files; // instantiates a vector of strings named
files that holds the filenames

    for (int i = 1; i < argc; ++i) files.push_back(argv[i]); // loops through
the command line arguments and adds each one to files, starts at 1 since
argv[0] is the program name

    if (files.empty()) // checks if no filenames were given

        for (int i = 1; i <= 5; ++i) files.push_back("puzzle" +
std::to_string(i) + ".txt"); // defaults to puzzle1.txt through puzzle5.txt



    SudokuApplication app; // creates the application that does all the work

    return app.run(files); // runs the application and returns its status

} // ends main function
```

Fixes

```
/*
* Program Name: EECS 348 Assignment 4
*
```

```
* Description: A program that solves sudoku puzzles using a recursive object
oriented searching method. It finds every solution showing puzzles that have
multiple solutions and showing puzzles with no solution
*
* Inputs: Text files that contain 9x9 sudoku puzzles, a row per line that
contain digits from 1-9 for filled pieces and _ for empty pieces. Uses the
files puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
*
* Outputs: The file name, the puzzle read from the file, and then the
solutions all written to the output
*
* Collaborators: Connor Wiley
*
* Other sources: Code generated via Claude Anthropic Sonnet 5.5
*
* Author: Connor Wiley, Claude Anthropic
*
* Creation Date: 10/15/2026
*/



// Sudoku brute-force solver (depth-first search with backtracking).

// Object-oriented design: all logic lives in classes; main() only creates

// an application object and runs it.

//

// Build:  g++ -std=c++17 -O2 -o sudoku sudoku.cpp

// Run:    ./sudoku puzzle1.txt puzzle2.txt puzzle3.txt puzzle4.txt
puzzle5.txt

//         (with no arguments it looks for puzzle1.txt ... puzzle5.txt in the

//          current directory)



#include <array> // allows usage of std::array, used as a fixed size for the
board

#include <cctype> // allows usage of character helper functions, used for
std::isspace when reading puzzles
```

```
#include <fstream> // allows usage of ifstream, used to open the puzzle files

#include <iostream> // allows usage of cout and ostream, used for printing
outputs like boards and messages

#include <string> // allows usage of the string type, used for filenames and
for reading the puzzle line by line

#include <utility> // allows usage of std::move, used to hand the solutions
back without copying them

#include <vector> // allows usage of vector type, used to hold solutions and
filenames



// ------------------------------------------------------------------------

// Board: a 9x9 grid of digits (0 = empty). Knows the Sudoku rules.

// ------------------------------------------------------------------------

class Board { // creates a class Board that holds grids and defines sudoku

public: // makes it so everything below can be called by outside functions

    static const int SIZE = 9; // instantiates an int named size for the width
and height of the board, static so it doesnt change

    static const int BLOCK = 3; // instantiates an int named block for the
width and height of each small box 3x3 for example

    static const int ALL_DIGITS = 0x3FE; // instantiates an int named
ALL_DIGITS that has the bits for digits 1 through 9 turned on (bit 0 is
unused), used to build the candidate mask



    Board() { // creates a constructor that fills each cell with zero so we
have an empty board

        cells_.fill(0); // fills every cell with 0

        rowMask_.fill(0); // clears the record of which digits are used in
each row
```

```
        colMask_.fill(0); // clears the record of which digits are used in
each column

        blockMask_.fill(0); // clears the record of which digits are used in
each 3x3 block

    } // ends constructor



    int get(int row, int col) const { return cells_[row * SIZE + col]; } //
creates the get function that returns the value at a row and column in the
array

    void set(int row, int col, int value) { // creates the set function that
puts a value at the specified row and column in the array and keeps the used
digit masks up to date

        int old = cells_[row * SIZE + col]; // instantiates an int named old
that holds the digit currently in the cell

        int b = (row / BLOCK) * BLOCK + col / BLOCK; // instantiates an int
named b that holds the number 0 through 8 of the 3x3 block the cell is in

        if (old != 0) { // checks if the cell currently holds a digit

            rowMask_[row] &= ~(1 << old); // removes the old digit from the
digits used in the row

            colMask_[col] &= ~(1 << old); // removes the old digit from the
digits used in the column

            blockMask_[b] &= ~(1 << old); // removes the old digit from the
digits used in the block

        } // ends if statement

        if (value != 0) { // checks if the new value is a digit and not a
blank

            rowMask_[row] |= (1 << value); // marks the new digit as used in
the row

            colMask_[col] |= (1 << value); // marks the new digit as used in
the column
```

```
            blockMask_[b] |= (1 << value); // marks the new digit as used in
the block

        } // ends if statement

        cells_[row * SIZE + col] = value; // stores the value in the cell

    } // ends set function



    // True if `value` can legally be placed at (row, col) given the other
cells.

    bool isCandidate(int row, int col, int value) const { // creates the
isCandiate function that checks if a spot is legal to enter

        for (int i = 0; i < SIZE; ++i) { // loops through all positions in the
row and columns to check them

            if (i != col && get(row, i) == value) return false;  // row //
checks the rows

            if (i != row && get(i, col) == value) return false;  // column //
checks the columns

        } // ends the for loop for columns and rows

        int br = (row / BLOCK) * BLOCK, bc = (col / BLOCK) * BLOCK; //
instantiates the top left corner of the 3x3 block the cells is in, rounding it
down

        for (int r = br; r < br + BLOCK; ++r) // loops through the 3 rows of
the block

            for (int c = bc; c < bc + BLOCK; ++c) // loops through the 3
columns of the block

                if ((r != row || c != col) && get(r, c) == value) return
false;  // block // checks the block for exact value, skipping the original
cell, if theres a match its invalid

        return true; // combonation is correct so value is allowed

    } // ends isCanidate function
```

```
    // Bitmask of the digits that may legally go into the EMPTY cell (row,
col), bit v is on if digit v is allowed.

    int candidateMask(int row, int col) const { // creates the candidateMask
function that replaces the old candidates function, it returns a bitmask
instead of building a vector

        int b = (row / BLOCK) * BLOCK + col / BLOCK; // instantiates an int
named b that holds the number 0 through 8 of the 3x3 block the cell is in

        int used = rowMask_[row] | colMask_[col] | blockMask_[b]; //
instantiates an int named used that combines every digit already used in the
row, column, and block

        return ~used & ALL_DIGITS; // flips the used digits so the legal ones
are on, then keeps only bits 1 through 9

    } // ends candidateMask function



    // Finds the next empty cell in row-major order starting at index `start`.
Returns false if there is none.

    bool findEmpty(int& row, int& col, int start = 0) const { // creates the
findEmpty function that finds the next empty cell, row and col are references
so the answer gets sent back through them, start lets the search skip cells
that are already known to be filled

        for (int i = start; i < SIZE * SIZE; ++i) { // loops through the cells
in order beginning at start instead of always at cell 0

            if (cells_[i] == 0) { // checks if the cell is empty

                row = i / SIZE; // converts the index into a row number

                col = i % SIZE; // converts the index into a column number

                return true; // returns true since an empty cell was found

            } // ends if statement

        } // ends for loop
```

```
        return false; // returns false since no empty cells are left, meaning
the board is full

    } // ends findEmpty function



    // True if no filled cell conflicts with another (used to vet the given
puzzle).

    bool isConsistent() const { // creates the isConsistent function that
checks if the numbers already on the board break any rules

        for (int r = 0; r < SIZE; ++r) // loops through each row

            for (int c = 0; c < SIZE; ++c) { // loops through each column

                int v = get(r, c); // instantiates an int named v that holds
the value at that cell

                if (v != 0 && !isCandidate(r, c, v)) return false; // checks
if the cell is filled and its value conflicts with another cell, if so returns
false

            } // ends the for loop for columns

        return true; // returns true since nothing conflicted

    } // ends isConsistent function



    void print(std::ostream& out) const { // creates the print function that
writes the board to whatever output stream it is given

        for (int r = 0; r < SIZE; ++r) { // loops through each row

            for (int c = 0; c < SIZE; ++c) { // loops through each column

                int v = get(r, c); // instantiates an int named v that holds
the value at that cell

                out << (v == 0 ? '_' : static_cast<char>('0' + v)) << ' '; //
prints an underscore if the cell is empty, otherwise prints the digit as a
character, then a space
```

```
            } // ends the for loop for columns

            out << '\n'; // moves to the next line after each row

        } // ends the for loop for rows

    } // ends print function



private: // makes it so everything below can only be used inside the class

    std::array<int, SIZE * SIZE> cells_; // instantiates an array of 81 ints
named cells_ that stores the whole board in one line

    std::array<int, SIZE> rowMask_; // instantiates an array of 9 ints named
rowMask_ where each one has a bit on for every digit used in that row

    std::array<int, SIZE> colMask_; // instantiates an array of 9 ints named
colMask_ where each one has a bit on for every digit used in that column

    std::array<int, SIZE> blockMask_; // instantiates an array of 9 ints named
blockMask_ where each one has a bit on for every digit used in that 3x3 block

}; // ends Board class



// ----------------------------------------------------------------------------

// PuzzleReader: loads a Board from a text file ('_' = blank).

// ----------------------------------------------------------------------------

class PuzzleReader { // creates a class PuzzleReader that loads a board from a
text file

public: // makes it so everything below can be called by outside functions

    // Returns true on success and fills `board`.

    bool read(const std::string& filename, Board& board) const { // creates
the read function that fills a board from a file and returns whether it worked

        std::ifstream in(filename); // instantiates an ifstream named in that
```

```
opens the file

        if (!in) return false; // checks if the file failed to open, if so
returns false

        // Correctness fix: the file is now checked line by line, any
character that isnt a digit, _, ., 0, or whitespace makes the read fail, every
row must have exactly 9 cells, and extra content after the 9th row also makes
the read fail instead of being accepted with no warning

        std::string line; // instantiates a string named line that holds one
line of the file at a time

        int rows = 0; // instantiates an int named rows that tracks how many
rows have been read

        bool firstLine = true; // instantiates a bool named firstLine that is
true until the first line has been handled, used to remove a byte order mark

        while (std::getline(in, line)) { // loops reading one full line at a
time until the file ends

            if (firstLine) { // checks if this is the first line of the file

                firstLine = false; // marks that the first line has been
handled

                if (line.size() >= 3 && static_cast<unsigned char>(line[0]) ==
0xEF && static_cast<unsigned char>(line[1]) == 0xBB && static_cast<unsigned
char>(line[2]) == 0xBF) line.erase(0, 3); // removes the 3 byte order mark
that some editors add to the start of a file

            } // ends if statement

            std::string cells; // instantiates a string named cells that holds
the line with all whitespace removed

            for (char ch : line) // loops through each character of the line

                if (!std::isspace(static_cast<unsigned char>(ch))) cells +=
ch; // keeps the character only if it isnt whitespace, this also removes the
carriage return from windows line endings

            if (cells.empty()) continue; // skips blank lines

            if (rows >= Board::SIZE) return false; // returns false since
```

```
there is extra content after the 9th row

            if (static_cast<int>(cells.size()) != Board::SIZE) return false;
// returns false since the row doesnt have exactly 9 cells

            for (int col = 0; col < Board::SIZE; ++col) { // loops through
each cell in the row

                char ch = cells[col]; // instantiates a char named ch that
holds the current character

                if (ch == '_' || ch == '.' || ch == '0') board.set(rows, col,
0); // sets the cell to 0 if the character represents a blank cell

                else if (ch >= '1' && ch <= '9') board.set(rows, col, ch -
'0'); // sets the cell to the digit, subtracting '0' converts the character
into an int

                else return false; // returns false since the character isnt
allowed

            } // ends for loop

            ++rows; // adds 1 to rows since a full row was read

        } // ends while statement

        return rows == Board::SIZE; // returns true only if exactly 9 rows
were read

    } // ends read function

}; // ends PuzzleReader class



// ----------------------------------------------------------------------

// Solver: recursive depth-first search with backtracking. Collects EVERY

// solution (the puzzle may not have a unique one).

// ----------------------------------------------------------------------

class Solver { // creates a class Solver that finds every solution using
backtracking
```

```
public: // makes it so everything below can be called by outside functions

    static constexpr std::size_t MAX_SOLUTIONS = 1000; // instantiates a
constant named MAX_SOLUTIONS that is the most solutions the solver will store
before it stops searching



    // True if the last solveAll stopped early because it hit MAX_SOLUTIONS.

    bool limitReached() const { return limitReached_; } // creates the
limitReached function that lets the application know the list of solutions was
cut off



    // Solves `puzzle` and returns all solutions found (empty => no solution).

    std::vector<Board> solveAll(const Board& puzzle) { // creates the solveAll
function that returns every solution to a puzzle

        solutions_.clear(); // clears any solutions left over from a previous
puzzle

        limitReached_ = false; // resets the limit flag from a previous puzzle

        if (!puzzle.isConsistent()) return solutions_;  // given digits
already clash // checks if the given numbers already break the rules, if so
returns the empty list since theres no solution

        Board work = puzzle; // instantiates a Board named work that is a copy
of the puzzle so the original doesnt change

        search(work, 0); // calls search on the copy, starting at cell 0, to
start finding solutions

        return std::move(solutions_); // moves the solutions out to the caller
instead of copying the whole vector

    } // ends solveAll function
```

```
private: // makes it so everything below can only be used inside the class

    // Space complexity fix: storing solutions is now capped at max_solutions
(1000 boards is about 324 KB) so a nearly empty puzzle cant exhaust memory,
and solveAll moves the vector out instead of copying it



    std::vector<Board> solutions_; // instantiates a vector of Boards named
solutions_ that holds each finished board

    bool limitReached_ = false; // instantiates a bool named limitReached_
that becomes true once the solution cap is hit

    // Execution time fix: the worst case is still O(9^m) for m empty cells,
but each node is much cheaper now. findEmpty starts scanning at the cell after
the last one filled instead of rescanning from cell 0, and the legal digits
come in one step instead, the search also stops as soon as the cap is hit.

    // Recursive step: fill the next empty cell with each candidate in turn.

    void search(Board& board, int start) { // creates the search function that
fills the next empty cell and then calls itself again, start is the index to
begin looking for an empty cell

        int row, col; // instantiates ints named row and col that hold the
location of the next empty cell

        if (!board.findEmpty(row, col, start)) {  // checks if there are no
empty cells left

            solutions_.push_back(board); // adds the finished board to the
list of solutions

            if (solutions_.size() >= MAX_SOLUTIONS) limitReached_ = true; //
checks if the cap was reached, if so sets the flag so the search stops

            return; // goes back to the previous call to try other options

        } // ends if statement

        int mask = board.candidateMask(row, col); // instantiates an int named
mask that holds a bit for every legal digit in that cell

        for (int value = 1; value <= Board::SIZE; ++value) {  // none => dead
```

```
end, backtrack // loops through the digits 1 through 9

            if (!(mask & (1 << value))) continue; // skips the digit if its
bit isnt on in the mask, meaning it isnt legal there

            board.set(row, col, value); // puts the digit in the cell

            search(board, row * Board::SIZE + col + 1);          // recurse
with one more cell filled // calls search again now that the cell is filled,
starting just after this cell since everything before it is filled

            board.set(row, col, 0); // undo and try the next candidate // sets
the cell back to empty so the next digit can be tried

            if (limitReached_) return; // stops trying digits once the cap was
reached

        } // ends for loop

    } // ends search function

}; // ends Solver class



// ----------------------------------------------------------------------------

// Application: drives the whole program (reading, solving, reporting).

// ----------------------------------------------------------------------------

class SudokuApplication { // creates a class SudokuApplication that runs the
whole program

public: // makes it so everything below can be called by outside functions

    int run(const std::vector<std::string>& files) { // creates the run
function that goes through each puzzle file and returns 1 if any of them
couldnt be read

        int status = 0; // instantiates an int named status set to 0, meaning
everything has worked so far

        for (const std::string& name : files) { // loops through each filename

            std::cout << "==============================\n"; // prints a line
```

```
to separate the puzzles

            std::cout << "Puzzle file: " << name << "\n\n"; // prints which
file is being used



            Board puzzle; // instantiates an empty Board named puzzle to read
the file into

            if (!reader_.read(name, puzzle)) { // checks if the file couldnt
be read into a valid board

                std::cout << "Could not read a valid puzzle from this
file.\n\n"; // prints an error message

                status = 1; // sets status to 1 so main knows something failed

                continue; // skips to the next file

            } // ends if statement



            std::cout << "Puzzle:\n"; // prints the label for the starting
board

            puzzle.print(std::cout); // prints the starting board

            std::cout << '\n'; // prints a blank line after the board



            std::vector<Board> solutions = solver_.solveAll(puzzle); //
instantiates a vector of Boards named solutions that holds everything the
solver finds

            if (solutions.empty()) { // checks if no solutions were found

                std::cout << "No solution found\n\n"; // prints that there is
no solution

            } else { // ends the if statement but runs if at least one
solution was found

                std::cout << solutions.size()
```

```
                        << (solutions.size() == 1 ? " solution found:\n\n" :
" solutions found:\n\n"); // prints how many solutions were found, using
singular or plural depending on the count

                for (std::size_t i = 0; i < solutions.size(); ++i) { // loops
through each solution

                    if (solutions.size() > 1) std::cout << "Solution " << (i +
1) << ":\n"; // checks if theres more than one solution, if so prints a
numbered label

                    else std::cout << "Solution:\n"; // otherwise prints the
plain label

                    solutions[i].print(std::cout); // prints the solved board

                    std::cout << '\n'; // prints a blank line after the board

                } // ends for loop

                if (solver_.limitReached()) std::cout << "Stopped after " <<
Solver::MAX_SOLUTIONS << " solutions, there may be more.\n\n"; // checks if
the solver hit its cap, if so prints a note so the list isnt mistaken for
every solution

            } // ends else statement

        } // ends for loop

        return status; // returns 0 if every file was read fine, otherwise 1

    } // ends run function



private: // makes it so everything below can only be used inside the class

    PuzzleReader reader_; // instantiates a PuzzleReader named reader_ that
loads the puzzle files

    Solver solver_; // instantiates a Solver named solver_ that solves the
puzzles

}; // ends SudokuApplication class
```

```
int main(int argc, char* argv[]) { // instantiates the main function for
running the program

    std::vector<std::string> files; // instantiates a vector of strings named
files that holds the filenames

    for (int i = 1; i < argc; ++i) files.push_back(argv[i]); // loops through
the command line arguments and adds each one to files, starts at 1 since
argv[0] is the program name

    if (files.empty()) // checks if no filenames were given

        for (int i = 1; i <= 5; ++i) files.push_back("puzzle" +
std::to_string(i) + ".txt"); // defaults to puzzle1.txt through puzzle5.txt



    SudokuApplication app; // creates the application that does all the work

    return app.run(files); // runs the application and returns its status

} // ends main function
```